

Problem 7.1

```
// Use Euclid's algorithm to calculate the GCD.

private long GCD( long a, long b )
{
    // Repeat until we're done
    for( ; ; )
    {
        // Set remainder to the remainder of a / b
        long remainder = a % b;

        // if remainder is 0, we're done.  Return b.
        if( remainder == 0 ) return b;

        // Set a = b and b = remainder.
        a = b;
        b = remainder;
    }
}
```

Problem 7.2

When you write comments before the code is finished and the code later changes, or when you copy comments from somewhere else without updating them to match the new code.

Problem 7.4

Yes, because the function receives input from outside the program and you cannot guarantee that the user will never pass in a zero value for `b`, which would cause a crash. Since you cannot control what values get passed in.

Problem 7.7

1. Get in the car and start the engine
2. Back out of the driveway
3. Turn onto the main road
4. Follow the route to the supermarket
5. Turn into the supermarket parking lot
6. Park the car and turn off the engine

Assumptions: The car is in working order and has gas, the driver knows the route, the road conditions are normal

Problem 8.1

```
def GCD(a, b):
    a = abs(a)
    b = abs(b)
    if a == 0: return b
    if b == 0: return a
    while True:
        remainder = a % b
        if remainder == 0: return b
        a = b
        b = remainder
```

```
def isRelativelyPrime(a, b):
    if a == 0: return b == 1 or b == -1
    if b == 0: return a == 1 or a == -1
    gcd = GCD(a, b)
    return gcd == 1 or gcd == -1
```

Test cases

```
print(isRelativelyPrime(8, 9)) # True - no common factors
print(isRelativelyPrime(21, 35)) # False - both divisible by 7
print(isRelativelyPrime(0, 1)) # True - 1 is relatively prime to 0
print(isRelativelyPrime(0, 5)) # False - 5 is not relatively prime to 0
print(isRelativelyPrime(1, 1000000)) # True - 1 is relatively prime to everything
print(isRelativelyPrime(-1, 1000000)) # True - -1 is relatively prime to everything
print(isRelativelyPrime(-21, -35)) # False - both divisible by 7
```

Problem 8.3

A) Gray-box testing because we have some knowledge of how it works internally

B) Exhaustive testing is possible but it will take a lot of time given the large range of the bounds. White box testing will also work testing the zero checks and the GCD loop

Problem 8.5

```
import math
```

```
def GCD(a, b):
    a = abs(a)
    b = abs(b)
    if a == 0: return b
    if b == 0: return a
    while True:
        remainder = a % b
        if remainder == 0: return b
        a = b
        b = remainder
```

```
def areRelativelyPrime(a, b):
    if a == 0: return b == 1 or b == -1
    if b == 0: return a == 1 or a == -1
    gcd = GCD(a, b)
    return gcd == 1 or gcd == -1
```

```
# Test cases
```

```
tests = [
    (8, 9, True),
    (21, 35, False),
    (0, 1, True),
    (0, -1, True),
    (0, 5, False),
    (1, 1000000, True),
    (-1, 1000000, True),
    (-21, -35, False),
]
```

```
for a, b, expected in tests:
```

```
    result = areRelativelyPrime(a, b)
    status = "Pass" if result == expected else "Fail"
    print(f'{status}: areRelativelyPrime({a}, {b}) = {result}, expected {expected}')
```

8.9

Exhaustive testing is black-box testing because it tests every possible input without any knowledge of the internal code. You are purely testing inputs and outputs, which is the definition of black-box testing.

Problem 8.11

Alice and Bob share: {2,5} — 2 bugs in common Lincoln estimate = $(5 \times 4) / 2 = 10$ bugs

Alice and Carmen share: {1,2} — 2 bugs in common Lincoln estimate = $(5 \times 5) / 2 = 12.5$ bugs

Bob and Carmen share: {2} — 1 bug in common Lincoln estimate = $(4 \times 5) / 1 = 20$ bugs

Average estimate = $(10 + 12.5 + 20) / 3 = \sim 14$ bugs total

10 unique bugs found so far, ~4 still at large.

Problem 8.12

If two testers find no bugs in common the formula divides by zero, which means the estimate is undefined. This suggests the two testers are finding completely different bugs and the total number of bugs is very large. The lower bound is the number of unique bugs